



Adapting to Environment Changes Through Neuromodulation of Reinforcement Learning

Jinwei Xing^{1(✉)}, Xinyun Zou¹, Praveen K. Pilly², Nicholas A. Ketz²,
and Jeffrey L. Krichmar¹

¹ University of California, Irvine, CA 92697, USA
{jinweix1,xinyunz5,jkrichma}@uci.edu

² HRL Laboratories, LLC, Malibu, CA 90265, USA
{pkpilly,naketz}@hrl.com

Abstract. Reinforcement learning (RL) enables agents to learn actions that can give maximum reward in an interactive environment. The environment is normally described as a predefined Markov Decision Process (MDP) which is assumed to remain unchanged throughout the life of RL agents. However, RL faces challenges when the environment changes. First, the agent needs to be able to detect the environment change rapidly when it occurs. Second, the agent needs to retain the knowledge learned before the environmental change. When facing an environment that was interacted with before, the learned knowledge should be recalled and utilized. To overcome these two challenges, we developed a biologically-inspired neuromodulation system that enables RL agents to quickly detect and adapt to environment changes. Our neuromodulation system is inspired by the effects of the cholinergic (ACh) and noradrenergic (NE) neuromodulatory systems on tracking uncertainties in the environment. We conducted experiments in the Gridworld environment and on a simulated MuJoCo robot to demonstrate the efficacy of our approach.

Keywords: Neuromodulation · Reinforcement learning · Uncertainty

1 Introduction

Reinforcement learning is a type of learning technique that enables agents to select the appropriate actions in an environment to maximize cumulative reward. In recent years, by combining RL with deep neural networks, deep RL has achieved success in a wide range of applications such as games [1,2], robot control [3,4] and mobile autonomous driving [5,6]. Despite these successes, RL still faces challenges when applied to more practical scenarios such as dynamic environments that contain uncertainty.

In this work, we investigate how an RL agent can cope with uncertainty in dynamic environments. There are two challenges for the agent. First, the agent

needs to detect the environment change rapidly. Without rapid detection, the knowledge learned before the change could be tainted by the different reward feedback received by the agent after the change. Furthermore, not recognizing the change could result in performance drops. Second, the agent needs to remember the knowledge learned under each environment setting. This requires the agent to not only detect the environment change but also identify whether the changed environment has been interacted with before. When facing a familiar environment setting, the agent needs to recall the knowledge and avoid learning from scratch again.

To address the challenges above, we take inspiration from humans and animals. Decision making is a core competency for humans and animals to survive in the environment. In the past several decades, neuroscience research in decision making tasks support the idea that the brain uses a form of reinforcement learning to shape decision making [7–10]. However, humans and animals face a similar problem of adapting to environment changes as the real world is normally uncertain. We suggest that brain’s neuromodulatory system plays an important role in coping with this uncertainty.

In prior work, we investigated how neuromodulated neural networks could rapidly adapt to goal changes in classification tasks [12]. It was based on a Bayesian model of neuromodulation to track the uncertainty of interactions with the environment [11] in which the cholinergic (ACh) system tracked expected uncertainty (i.e., the known degree of unreliability of predictive relationships in the environment) and the noradrenergic (NE) system tracked unexpected uncertainty (i.e., large changes in the environment that violate prior expectations). In addition, the NE system caused a rapid adaptation to goal changes by triggering a ‘network reset’ [16, 17]. The present work extends this goal-driven perception model to RL agents, which must recognize changes to their reward function and adapt appropriately.

Inspired by the neuromodulation system described above, we developed a reinforcement learning system in which ACh system tracks the expected uncertainty of the current reward function while NE system tracks the unexpected uncertainty which could increase when prior actions no longer produce rewards. These two systems allow the agent to detect task changes rapidly and identify whether a task is novel or not and thus decide whether and which knowledge should be recalled. We show how this approach can improve the performance of RL agents in a Gridworld environment and a MuJoCo walking robot.

2 Problem

In this work, we focus on the problem of how to enable reinforcement learning agents to rapidly adapt to changes in the environment. We first introduce the basic setting of reinforcement learning and then explain the form of environmental changes used in the present work.

2.1 Reinforcement Learning

In reinforcement learning, agents learn to take actions to maximize their cumulative rewards in the environment. The environment is typically stated in the form of a Markov Decision Process (MDP), which is expressed in terms of the tuple (S, A, T, R) where S is the state space, A is the action space, T is the state transition function and R is the reward function. At each time step t in the MDP, the agent takes an action a_t in the environment based on current state s_t and receives a reward r_{t+1} following R and next state s_{t+1} following T . The state transition and reward feedback follows:

$$s_{t+1} = T(s_t, a_t) \quad (1)$$

$$r_t = R(s_t, a_t) \quad (2)$$

The goal of RL agent is to find a policy $\pi(s)$ to select actions that maximize the discounted cumulative future reward $r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$, where γ is the discount factor ranging from 0 to 1.

2.2 Environment Changes

In this work, we focus on how to adapt when feedback from the environment changes the reward function R while other elements including state space S , action space A and state transition function T remain unchanged. Here we define a task as a MDP:

$$Task_i = \langle S, A, T, R_i \rangle \quad (3)$$

where i is the identity of the task. All tasks share the same $\langle S, A, T \rangle$ while each task has its own R_i . The environment change could be demonstrated as task switching in a task sequence. As noted below, we define the environment to be composed of a sequence of tasks.

$$Env = [Task_1, Task_2, Task_3, \dots, Task_N] \quad (4)$$

where N is the number of tasks in the sequence. Note that $Task_i$ and $Task_j$ may share the same reward function, in which case $R_i = R_j$. This means a given task can occur multiple times in the sequence, and so the agent needs to learn the task and re-utilize the learned knowledge when exposed to the task again.

3 Method

We developed a bio-inspired neuromodulatory system to track the uncertainty of the environment which facilitates rapid adaptation to environment changes. Similar to the model in [12], our system is composed of ACh and NE neuromodulatory systems. We introduce their underlying equations below.

3.1 ACh and NE Neuromodulation

In our system, the ACh system has K ACh neurons where K is the number of tasks the agent has detected and initialized as 1. As more tasks are detected by our neuromodulatory system, the value of K increases accordingly. We use a vector to represent the activity of ACh neurons, where ACh_i represents the uncertainty of $Task_i$. The higher ACh_i is, the more certain that the current task is $Task_i$. A task change is detected when the maximum ACh value is below a threshold ch_{change} .

$$Task_Change = \begin{cases} True & \text{if } \max(ACh) < ch_{change} \\ False & \text{otherwise} \end{cases} \quad (5)$$

Once a task change is detected, the agent needs to judge whether the new task is a task that has been learned before and thus previous knowledge should be recalled or the new task is novel and needs to be learned from scratch. We call this step a “task match”. A task match should happen when the agent has low uncertainty on one task and high uncertainty on all other tasks, as described below.

$$P_i = \frac{\exp(\beta * ACh_i)}{\sum_j^K \exp(\beta * ACh_j)} \quad (6)$$

$$Task_Match_i = \begin{cases} True & \text{if } P_i > p_{match} \\ False & \text{otherwise} \end{cases} \quad (7)$$

where P_i represents the certainty that the new task is $Task_i$. Once P_i goes above the threshold of p_{match} , the system matches the new task with $Task_i$ and $Task_Match_i$ is set as True. The softmax function in Eq. 6 allows the agent to take the complete ACh system into consideration in task match.

In addition to the task change detection and task match based on ACh system, we use the NE system to decide whether a new task is novel. The NE system consists of one NE neuron whose activity represents the unexpected uncertainty on the environment. The activity of the NE neuron increases when a task change is detected by the ACh system while the new task cannot be matched with previous tasks. When the activity of the NE neuron goes above a hyperparameter threshold $ne_{threshold}$, the new task is believed to be novel.

$$Task_Novel = \begin{cases} True & \text{if } NE > ne_{threshold} \\ False & \text{otherwise} \end{cases} \quad (8)$$

3.2 Update of ACh and NE System

Since the only difference between tasks is the reward function R , the uncertainty about the environment and the update of our neuromodulatory system are based on the difference between the reward feedback expected by the agent and the

actual reward, which is called reward prediction error in this work. To produce the reward expectation, the agent learns a reward predictor $\tilde{R}_i$ to approximate R_i for each $Task_i$. For a set of $\langle s, a, r \rangle$, the reward prediction error is defined as the difference between the predicted reward $\tilde{R}_i(s, a)$ and the true reward r is

$$RPE_i = \tilde{R}_i(s, a) - r \quad (9)$$

The expected uncertainty (ACh neuron activity) increases when the reward prediction error is high and decreases when the reward prediction error is low. To simulate this, we use RPE_{mean} and RPE_{std} to track the running mean and standard deviation, respectively, of the reward prediction error and update the ACh system as follows:

$$Expected_i = \begin{cases} True & \text{if } abs(RPE_i - RPE_{mean}) < k * RPE_{std} \\ False & \text{otherwise} \end{cases} \quad (10)$$

$$ACh_i = \begin{cases} min(ch_{max}, ACh_i * ch_{expected}) & \text{if } Expected_i \\ max(ch_{min}, ACh_i * ch_{unexpected}) & \text{otherwise} \end{cases} \quad (11)$$

where $Expected_i$ represents whether the reward for $Task_i$ is expected by the agent. If so, ACh_i will increase. Otherwise, it will decrease. The hyperparameter k in Eq. 10 controls the strictness of $Expected_i$. $ch_{expected}$ and $ch_{unexpected}$ are scaling factors used to increase or decrease ACh neuron activities while ch_{max} and ch_{min} represents their maximum and minimum values.

The NE system tracks the unexpected uncertainty of the environment and can be used to detect novel tasks that have not been observed before. It is updated when a task change is detected and increases when the task match is incorrect. If the current task is matched with an old task, then the unexpected uncertainty is resolved and the NE neuron is reset.

$$NE = \begin{cases} min(ne_{max}, NE * ne_{unmatched}) & \text{if not any } Task_Match \\ ne_{init} & \text{otherwise} \end{cases} \quad (12)$$

where $ne_{unmatched}$ is a hyperparameter bigger than 1 that is used to increase the NE neuron activity. ne_{max} and ne_{init} represent the maximum and initial values of the NE neuron.

3.3 The Complete System

The complete system of our work includes an agent conducting reinforcement learning and a neuromodulatory system that helps it track the uncertainty of the environment and adapt to environment changes. The agent needs to remember the knowledge learned in each task. When a task is encountered again, the agent needs to conduct a task match based on the neuromodulatory system

and reactivate the knowledge once it is matched to avoid learning from scratch again. In this work, the knowledge includes the RL policy and reward predictor that the agent learns for each task. We describe the complete system with the pseudocode below.

Algorithm 1: Reinforcement Learning with Neuromodulatory System

Input: $ch_{init}, ch_{max}, ch_{min}, ch_{expected}, ch_{unexpected}, \beta, k, ne_{max}, ne_{init}, ne_{threshold}$
 $ne_{unmatched}, max_step$

Init: $task \leftarrow 1, ACh_1 \leftarrow ch_{init}, NE \leftarrow ne_{init}, task_change \leftarrow False, step \leftarrow 0, K \leftarrow 1$

```

1  while  $step < max\_step$  do
2    Agent selects action and receives feedback from the environment
3     $step \leftarrow step + 1$ 
4    if not  $Task\_Change$  then
5      Agent stores experiences for RL training
6      Compute the reward predictor error  $RPE_{task}$  // Equation 9
7      Update ACh system // Equations 10,11
8      Update  $Task\_Change$  based on ACh system // Equation 5
9      if not  $Task\_Change$  then
10       Update  $RPE_{mean}$  and  $RPE_{std}$ 
11       Train the RL agent
12       Train the reward predictor
13     else
14       Save the learned knowledge for  $task$ 
15   else
16     for  $i \leftarrow 1$  to  $K$  do
17       Compute  $Task\_Match_i$  // Equation 6, 7
18       if  $Task\_Match_i = True$  then
19          $task \leftarrow i$ 
20          $Task\_Change \leftarrow False$ 
21         reactivate the saved knowledge for task i
22         reset neuromodulation system
23       Update NE system // Equation 12
24       Compute  $Task\_Novel$  // Equation 8
25       if  $Task\_Novel = True$  then
26          $K \leftarrow K+1$ 
27          $task \leftarrow K$ 
28          $Task\_Change \leftarrow False$ 
29         create a new policy and reward predictor for the novel task
30         reset neuromodulation system

```

4 Experiments

We conduct two experiments to demonstrate the efficacy of our neuromodulatory system for RL applications: 1. Gridworld (Fig. 1) and 2. bipedal walking with a MuJoCo robot (Fig. 2).

The first experiment is based on a Gridworld Environment [13]. In this grid-based environment, there are four objects with unique colors where the red color represents the agent while the green, blue and yellow colors represent objects that can be picked up. The agent needs to navigate in the grid world and pick up the correct object. Based on the target object to pick up, we define three tasks named as pickup-green, pickup-blue and pickup-yellow. The agent receives non-zero reward only when picking up an object. The reward is 1 if the agent picks up the correct object and is -1 if the wrong object is picked up.

The second experiment is conducted on a MuJoCo simulated bipedal walker robot. DeepMind control suite [18] contains three walker-based tasks including walker-stand, walker-walk and walker-run. In the walker-stand task, the reward is a combination of terms encouraging an upright torso and some minimal torso height. The walker-walk and walker-run tasks include a component encouraging forward velocity. We list the hyperparameters in two experiments in Table 1.

Table 1. Hyperparameters of ACh and NE neuromodulatory systems.

Hyperparameters	GridWorld	Worker robot
ch_{init}	0.5	0.5
ch_{max}	1.0	1.0
ch_{min}	0.1	0.1
ch_{change}	0.2	0.2
$ch_{expected}$	1.1	1.1
$ch_{unexpected}$	0.9	0.9
p_{match}	0.5	0.5
β	2	2
ne_{max}	1.0	1.0
ne_{init}	0.1	0.1
$ne_{unmatched}$	1.05	1.05
$ne_{threshold}$	0.9	0.9

In our experiments, the environment change is demonstrated as a task switch. For Gridworld, we set the task sequence for the experiment of Gridworld as [pickup-green, pickup-blue, pickup-yellow, pickup-green, pickup-blue, pickup-yellow]. For the MuJoCo robot, we set the task sequence for the walker as [walker-stand, walker-walk, walker-run, walker-stand, walker-walk, walker-run]. The task switch in each sequence requires the agent to quickly detect environment changes

while the recurrence of tasks requires the agent to achieve successful task match and re-utilize learned knowledge.

Our neuromodulatory system is compatible with different types of reinforcement learning algorithms and settings. In the Gridworld experiments, the agent has a discrete action space and we use the [Proximal Policy Optimization \(PPO\) algorithm](#) [15], which is an on-policy RL method. In the MuJoCo walker robot experiment, the agent has a continuous action space and we use the [Twin Delayed Deep Deterministic policy gradient \(TD3\)](#) [14], which is an off-policy RL method.

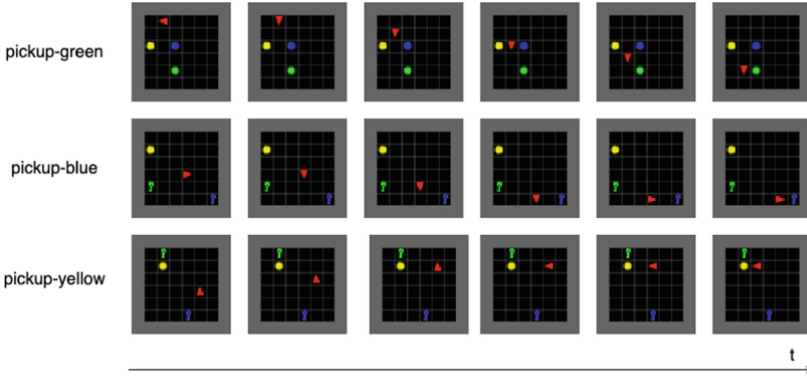


Fig. 1. Gridworld environment. Examples of three tasks (pickup-green, pickup-blue and pickup-yellow) in Gridworld environment. In each task, the agent moves towards the object with a specified color. (Color figure online)



Fig. 2. MuJoCo environment. Examples of three tasks (stand, walk and run) of MuJoCo walker robot. The agent has unique behavior in each task.

5 Results

5.1 Reinforcement Learning Performance

In both experiments, the addition of our neuromodulatory system improved performance. To demonstrate the efficacy of our method, we conducted an ablation study by disabling the NE system or disabling both the ACh and the NE system. Since the NE system depends on the ACh system, we avoided the setting of disabling the ACh while keeping the NE system. We compared the performance of the RL agent with and without ablations (Fig. 3). The better performance on novel tasks when neuromodulation was included due to the agent detecting environment changes quickly and avoiding detrimental knowledge transfer between tasks. Meanwhile, the immediate high score on recurring tasks demonstrates that the neuromodulatory system allows the agent to recognize previously observed tasks and utilize prior knowledge. In the ablation experiments, when the NE system is disabled, the agent cannot correctly identify whether a new task after task change is novel or not. As a result, all new tasks were simply recognized as novel tasks and the agent conducted learning from scratch every time a task change was detected. When the ACh system is also removed, the agent cannot detect task changes and conduct normal RL training throughout the whole task sequence. As a result, the agent would have only one task policy and the knowledge learned in the previous task will all be transferred to the next one. Its influence on the reward performance depends on the similarities of tasks in the task sequence. For example, the knowledge transfer in Gridworld is negative since the correct object to pick up in the last task will be wrong in the next task. As for tasks on walker robot, knowledge transfer could be more positive since learning to run could benefit from the knowledge of how to walk. However, in both two experiments, the RL agent with our neuromodulatory system achieved the best performance.

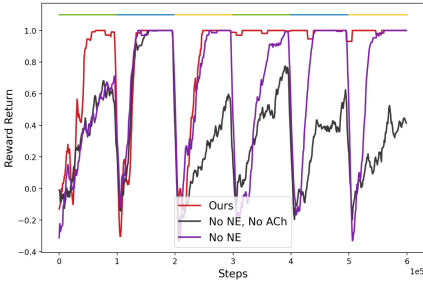
Besides the general RL performance, we're also interested in the benefits of our neuromodulatory system in fast performance recovery. In the experiments above, each task occurs multiple times in the task sequence and the agent gradually improves performance via learning when interacting with each task. Although task changes regularly following the task sequence, an intelligent agent should quickly recover the performance when encountering the same task again. As a result, we compute the average time steps needed to recover 90% of the performance when facing a task that has been learned before. As shown in Table 2, our neuromodulatory system allows the agent to achieve much faster performance recovery compared to the ablated agents.

5.2 Activity of Neuromodulatory System

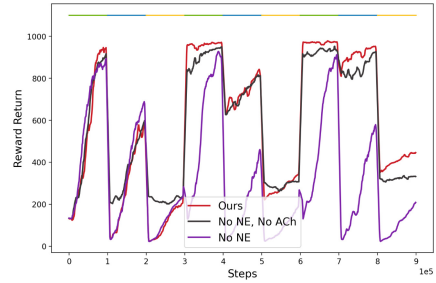
We examined the activity of the ACh and NE systems in these two experiments. As shown in Fig. 4, our ACh system is able to track the uncertainty on each task and always show high certainty on the correct task. This allows the agent to efficiently identify the correct task identity and adapt to task switches.

Table 2. Average time steps needed to achieve 90% performance recovery of each task of our method and ablated studies. The results are averaged over 6 runs.

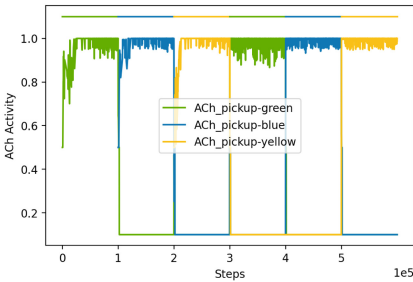
	Time steps of performance recovery		
	Ours	No NE	No NE, No ACh
pickup-green	150.3	45060.5	59753.3
pickup-blue	130.7	25510.7	69535.2
pickup-yellow	143.3	29428.7	87605.6
walker-stand	796.0	311000.0	354500.0
walker-walk	1843.0	411500.0	598000.0
walker-run	1542.2	501000.0	585333.3



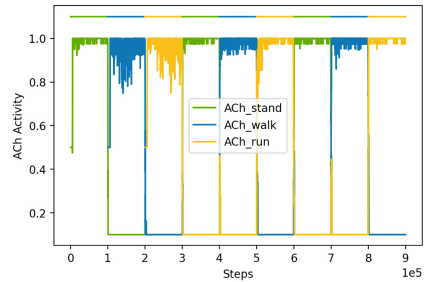
(a) RL Performance on Gridworld



(b) RL Performance on walker Robot

Fig. 3. Results of reinforcement learning performance with and without the neuromodulatory system. The bar on top of the figure represents the task sequence. The task sequence in Gridworld experiment is [pickup-green, pickup-blue, pickup-yellow, pickup-green, pickup-blue, pickup-yellow] and the task sequence in walker robot experiment is ['walker-stand', 'walker-walk', 'walker-run', 'walker-stand', 'walker-walk', 'walker-run', 'walker-stand', 'walker-walk', 'walker-run']. (Color figure online)

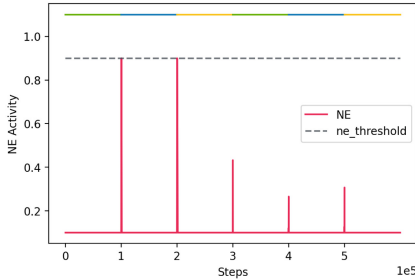
(a) ACh activity for the Gridworld experiment.



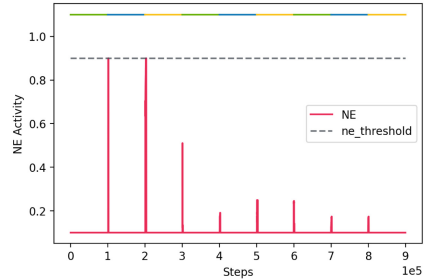
(b) ACh activity for the walker robot experiment.

Fig. 4. Dynamics of the activity of the ACh system in the two experiments. The bar on top of the figure represents the task sequence.

As shown in Fig. 5, our NE system shows high activity for the first and second task switches since the second and third tasks are novel. The activity of the NE system also increased for other task switches but didn't reach the threshold because those switched tasks had been encountered before by the agent and were not novel. This demonstrates that our NE system can track the unexpected uncertainty of the environment which allows the agent to distinguish between novel and familiar tasks. This capability helps the agent to better adapt to environment changes.



(a) NE activity for the Gridworld experiment.



(b) NE activity for the walker robot experiment

Fig. 5. Dynamics of the activity of the NE system in the two experiments. The bar on top of the figure represents the task sequence.

6 Conclusion

We developed a system inspired by neuromodulation to track the uncertainty of the environment and help reinforcement learning agents to quickly adapt to environment changes. We demonstrated the efficacy of the neuromodulatory system in two reinforcement learning experiments; namely, a Gridworld environment and a simulated walker robot. We believe these results provide insights into how intelligent agents survive in uncertain environments and also enable the deployment of artificial agents in complicated real-world applications.

Acknowledgements. This material is partially based upon work supported by the United States Air Force and DARPA under Contract No. FA8750-18-C-0103. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Air Force and DARPA. Authors are also thankful to computing resources provided by CHASE-CI under NSF Grant CNS-1730158.

References

1. Silver, D., et al.: Mastering the game of Go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (2016)

2. Mnih, V., et al.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015)
3. Akkaya, I., et al.: Solving Rubik’s cube with a robot hand. arXiv preprint [arXiv:1910.07113](https://arxiv.org/abs/1910.07113) (2019)
4. Lillicrap, T.P., et al.: Continuous control with deep reinforcement learning. arXiv preprint [arXiv:1509.02971](https://arxiv.org/abs/1509.02971) (2015)
5. Kahn, G., Abbeel, P., Levine, S.: BADGR: an autonomous self-supervised learning-based navigation system. *IEEE Robot. Autom. Lett.* **6**(2), 1312–1319 (2021)
6. Xing, J., Zou, X., Krichmar, J.L.: Neuromodulated patience for robot and self-driving vehicle navigation. In: 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE, July 2020
7. Montague, P.R., Dayan, P., Sejnowski, T.J.: A framework for mesencephalic dopamine systems based on predictive Hebbian learning. *J. Neurosci.* **16**(5), 1936–1947 (1996)
8. Schultz, W., Dayan, P., Montague, P.R.: A neural substrate of prediction and reward. *Science* **275**(5306), 1593–1599 (1997)
9. Hare, T.A., O’Doherty, J., Camerer, C.F., Schultz, W., Rangel, A.: Dissociating the role of the orbitofrontal cortex and the striatum in the computation of goal values and prediction errors. *J. Neurosci.* **28**(22), 5623–5630 (2008)
10. Parker, N.F., et al.: Reward and choice encoding in terminals of midbrain dopamine neurons depends on striatal target. *Nat. Neurosci.* **19**(6), 845–854 (2016)
11. Angela, J.Y., Dayan, P.: Uncertainty, neuromodulation, and attention. *Neuron* **46**(4), 681–692 (2005)
12. Zou, X., Kolouri, S., Pilly, P.K., Krichmar, J.L.: Neuromodulated attention and goal-driven perception in uncertain domains. *Neural Netw.* **125**, 56–69 (2020)
13. Chevalier-Boisvert, M., Willems, L., Pal, S.: Minimalistic gridworld environment for OpenAI Gym. GitHub Repository (2018). <https://github.com/maximecb/gym-minigrid>
14. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning, pp. 1587–1596. PMLR, July 2018
15. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint [arXiv:1707.06347](https://arxiv.org/abs/1707.06347) (2017)
16. Grella, S.L., et al.: Locus coeruleus phasic, but not tonic, activation initiates global remapping in a familiar environment. *J. Neurosci.* **39**(3), 445–455 (2019)
17. Bouret, S., Sara, S.J.: Network reset: a simplified overarching theory of locus coeruleus noradrenaline function. *Trends Neurosci.* **28**(11), 574–582 (2005)
18. Tassa, Y., et al.: Deepmind control suite. arXiv preprint [arXiv:1801.00690](https://arxiv.org/abs/1801.00690) (2018)